Introduction to Database Systems & Relational Databases

Database

- A ***database*** is a collection of related data stored in a computer.
- The data in a database is typically organized into ***records***.

STUDENT	SId	SName	GradYear	MajorId
	1	joe	2004	10
	2	amy	2004	20
	3	max	2005	10
	4	sue	2005	20
	5	bob	2003	30
	6	kim	2001	20
	7	art	2004	30
	8	pat	2001	20
	9	lee	2004	10

DEPT	DId	DName
	10	compsci
	20	math
	30	drama

COURSE	CId	Title	DeptId
	12	db systems	10
	22	compilers	10
	32	calculus	20
	42	algebra	20
	52	acting	30
	62	elocution	30

SECTION	SectId	CourseId	Prof	YearOffered
	13	12	turing	2004
	23	12	turing	2005
	33	32	newton	2000
	43	32	einstein	2001
	53	62	brando	2001

ENROLL	EId	StudentId	SectionId	Grade
	14	1	13	A
	24	1	43	C
	34	2	43	B+
	44	4	33	B
	54	4	53	A
	64	6	53	A

Figure 1-1
Some records for a university database

University Database

- Five types of records
- A conceptual picture of some records
  - Does not indicate:
    - How the records are stored
    - How they are accessed

STUDENT	SId	SName	GradYear	MajorId
	1	joe	2004	10
	2	amy	2004	20
	3	max	2005	10
	4	sue	2005	20
	5	bob	2003	30
	6	kim	2001	20
	7	art	2004	30
	8	pat	2001	20
	9	lee	2004	10

DEPT	DId	DName
	10	compsci
	20	math
	30	drama

COURSE	CId	Title	DeptId
	12	db systems	10
	22	compilers	10
	32	calculus	20
	42	algebra	20
	52	acting	30
	62	elocution	30

SECTION	SectId	CourseId	Prof	YearOffered
	13	12	turing	2004
	23	12	turing	2005
	33	32	newton	2000
	43	32	einstein	2001
	53	62	brando	2001

ENROLL	EId	StudentId	SectionId	Grade
	14	1	13	A
	24	1	43	C
	34	2	43	B+
	44	4	33	B
	54	4	53	A
	64	6	53	A

Figure 1-1
Some records for a university database

Database System

- A ***database system*** is a software that manages the records in a database.
- Requirements:
  - Must be persistent
  - Can be very large
  - Get shared
  - Must be kept accurate
  - Must be usable
- A ***database administrator*** (or DBA) is the user who is responsible for the smooth operation of the database system.

STUDENT	SId	SName	GradYear	MajorId
	1	joe	2004	10
	2	amy	2004	20
	3	max	2005	10
	4	sue	2005	20
	5	bob	2003	30
	6	kim	2001	20
	7	art	2004	30
	8	pat	2001	20
	9	lee	2004	10

DEPT	DId	DName
	10	compsci
	20	math
	30	drama

COURSE	CId	Title	DeptId
	12	db systems	10
	22	compilers	10
	32	calculus	20
	42	algebra	20
	52	acting	30
	62	elocution	30

SECTION	SectId	CourseId	Prof	YearOffered
	13	12	turing	2004
	23	12	turing	2005
	33	32	newton	2000
	43	32	einstein	2001
	53	62	brando	2001

ENROLL	EId	StudentId	SectionId	Grade
	14	1	13	A
	24	1	43	C
	34	2	43	B+
	44	4	33	B
	54	4	53	A
	64	6	53	A

Figure 1-1
Some records for a university database

Record Storage

- Databases must be persistent.
- Storing database records in text files:
  - The simplest and most straightforward approach
  - One file per record type
  - Each record could be a line of text, with its values separated by tabs

1 [TAB] j o e [TAB] 2 0 0 4 [TAB] 1 0 [RET] 2 [TAB] a m y [TAB] 2 0 0 4 [TAB] 2 0 [RET] ...

Figure 1-2
Implementing the STUDENT records in a text file

Record Storage: Text Files

- Disadvantages:
  - Updating the file takes too much time
  - Searching the file takes too much time

1 TAB j o e TAB 2 0 0 4 TAB 1 0 RET 2 TAB a m y TAB 2 0 0 4 TAB 2 0 RET ...

Figure 1-2
Implementing the STUDENT records in a text file

Data Models and Schemas

- Two different ways of expressing the university database:
  - As several collections of records
  - As several files
- Each of these ways can be specified as a ***schema*** in a ***data model***.

STUDENT	SId	SName	GradYear	MajorId
	1	joe	2004	10
	2	amy	2004	20
	3	max	2005	10
	4	sue	2005	20
	5	bob	2003	30
	6	kim	2001	20
	7	art	2004	30
	8	pat	2001	20
	9	lee	2004	10

DEPT	DId	DName
	10	compsci
	20	math
	30	drama

COURSE	CId	Title	DeptId
	12	db systems	10
	22	compilers	10
	32	calculus	20
	42	algebra	20
	52	acting	30
	62	elocution	30

SECTION	SectId	CourseId	Prof	YearOffered
	13	12	turing	2004
	23	12	turing	2005
	33	32	newton	2000
	43	32	einstein	2001
	53	62	brando	2001

ENROLL	EId	StudentId	SectionId	Grade
			13	A
			43	C
			43	B+
			33	B
			53	A
			53	A

1 [TAB] j o e [TAB] 2 0 0 4 [TAB] 1 0 [RET] 2 [TAB] a m y [TAB] 2 0 0 4 [TAB] 2 0 [RET] ...

Figure 1-2
Implementing the STUDENT records in a text file

Data Models and Schemas

- A ***data model*** is a framework for describing the structure of databases.
  - Relational model
  - File-system model
- A ***schema*** is the structure of a particular database.
  - Schemas in relational data model are expressed in terms of tables of records.
  - Schemas in file-system data model are expressed in terms of files of records.

Data Models and Schemas (Cont.)

File-system data model

- Student records are stored in the text file “student.txt”.
- There is one record per line.
- Each record contains four values, separated by tabs, denoting the student ID, name, graduation year, and major ID.
- Programs that read (and write to) the file are responsible for understanding and decoding this representation.

Relational data model

- The records are stored in a table named STUDENT.
- Each record contains four fields: an integer *SId*, a string *SName*, and integers *GradYear* and *MajorId*.
- Users access a table in terms of its records and fields: They can insert new records into a table, and retrieve, delete, or modify all records satisfying a specified predicate.

```
public static List<String> getStudents1997() {
      List<String> result = new ArrayList<String>();
      FileReader rdr = new FileReader("students.txt");
      BufferedReader br = new BufferedReader(rdr);
      String line = br.readLine();
      while (line != null) {
            String[] vals = line.split("\t");
            String gradyear = vals[2];
            if (gradyear.equals("1997"))
                  result.add(vals[1]);
            line = br.readLine();
      }
      return result;
}
```

(a) Using a file system model

```
select SName from STUDENT where GradYear = 1997
```

(b) Using the relational model

Figure 1-3
Two ways to retrieve the name of students graduating in 1997

Data Models and Schemas (Cont.)

- Most of the Java code deals with decoding the file:
  - Reading each record from the file
  - Splitting it into an array of values to be examined (Figure 1-3 (a))
- The SQL code only specifies the values to be extracted from the table
  - It says nothing about how to retrieve them (Figure 1-3 (b))

Data Models and Schemas (Cont.)

- Relational model – a ***conceptual model***
  - Its schemas are specified and manipulated without any knowledge of (and without caring) how it is to be implemented.
  - ***Conceptual schema*** describes what the data “is.”
- File-system model – a ***physical model***
  - Its schemas are specified and manipulated in terms of a specific implementation.
  - ***Physical schema*** describes how the database is implemented.

The Three Schema Levels

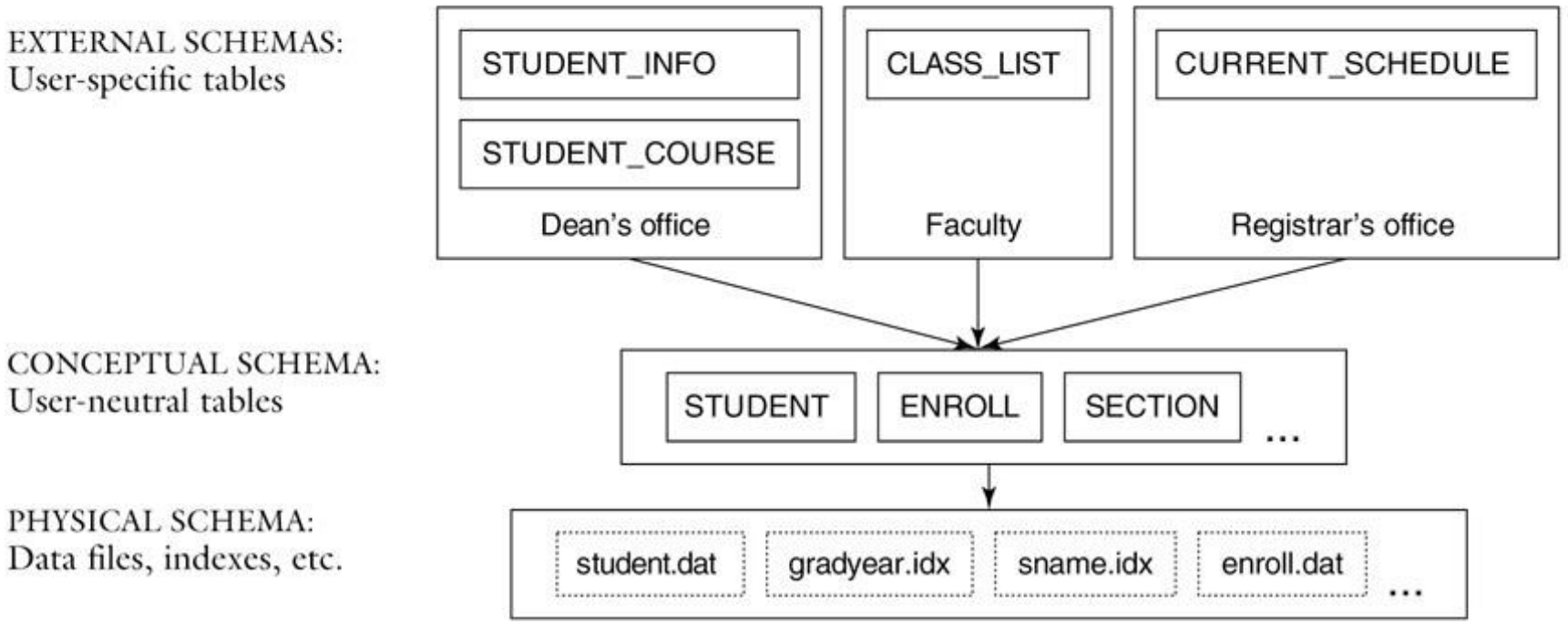

Figure 1-4
The three schema levels

Physical Data Independence

- DBMS translates operations on a conceptual schema into operations on a physical schema.
- A database system supports ***physical data independence*** if users do not need to interact with the database system at the physical level.
- Benefits:
  - Ease of use
  - Query optimization
  - Isolation from changes to the physical schema

Logical Data Independence

- The set of tables personalized for a particular user is called the user's ***external schema***.
- A database system supports ***logical data independence*** if users can be given their own external schema.
- Benefits:
  - Customized external schema
  - Isolation from changes to the conceptual schema
  - Better security

Relational Databases

Tables

- The data in a relational database system is organized into ***tables***.

```
STUDENT(SId, SName, GradYear, MajorId)
DEPT(DId, DName)
COURSE(CId, Title, DeptId)
SECTION(SectId, CourseId, Prof, YearOffered)
ENROLL(EId, StudentId, SectionId, Grade)
```

Figure 2-1
The schema of the university database

Null Values

- A ***null*** value denotes a value that does not exist or is unknown.
- Null values occur for two reasons:
  - Data collection may be incomplete.
  - Data may arrive late.

Superkeys and Keys

- A user must reference a record by specifying field values.
  - Example: “I want the record for student named Joe who graduated in 1977”.
- However, not all field values are guaranteed to uniquely identify a record.
- A unique identifier is called a ***superkey***.
- A ***superkey*** of a table is a field (or fields) whose values uniquely identify the table’s records.
- Note that adding a field to a superkey always produces another superkey.
- A ***key*** is a superkey having property that no subset of its fields is a superkey.

Superkeys and Keys

- STUDENT (SId, SName, GradYear, MajorId)
  - Every student has a different ID, so SId is a superkey of STUDENT.
  - SId is a key.
  - {SId, GradYear} is a superkey of STUDENT.

- SECTION (SectId, CourseId, Prof, YearOffered)
  - If a professor teaches at most one section of a given course in a year, then {Prof, YearOffered} is a key.
  - If a course can have at most one section per year, then {CourseId, YearOffered} is a key.

Primary Key

- Although a table can have several keys, one key is chosen to be the ***primary key***.
- Records are referenced by their primary key.
  - Each primary key should be as natural and easy to understand.
- ID numbers are often used as primary keys because they are simple and intuitive.
- Primary key fields must never be null.

Foreign Key

```
STUDENT(SId, SName, GradYear, MajorId)
DEPT(DId, DName)
COURSE(CId, Title, DeptId)
SECTION(SectId, CourseId, Prof, YearOffered)
ENROLL(EId, StudentId, SectionId, Grade)
```

Figure 2-1
The schema of the university database

- The information in a database is split among its tables.
- However, these tables are not isolated from each other.
- A **foreign key** is a field (or fields) of one table which corresponds to the primary key of another table.

MajorId	in STUDENT	is a foreign key of DEPT;
DeptId	in COURSE	is a foreign key of DEPT;
CourseId	in SECTION	is a foreign key of COURSE;
StudentId	in ENROLL	is a foreign key of STUDENT;
SectionId	in ENROLL	is a foreign key of SECTION.

Figure 2-2
Foreign keys for the university database

Foreign Keys and Referential Integrity

- The specification of a foreign key asserts ***referential integrity***, which requires each non-null foreign key value to be the key value of some record.

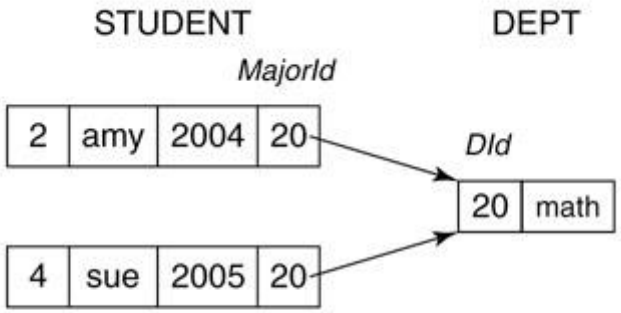

Figure 2-3
An example of referential integrity

Constraints

- A ***constraint*** describes the allowable states that the tables in the database may be in.
- There are four important kinds of a constraint:
  - ***Null value constraints*** specify that a particular field must not contain nulls.
  - ***Key constraints specify*** that two records cannot have the same values for the key's fields.
  - ***Referential integrity constraints*** specify that a foreign key value of one record must be the key value of another record.
  - ***Integrity constraints***

Integrity Constraints

- An ***integrity constraint*** encodes “business rules” about the organization.
- Integrity constraints have two purposes:
  - They can detect bad data entry.
  - They can enforce the “rules” of the organization.
- An integrity constraint may apply to
  - an individual record, “a student’s graduation year is at least 1863”
  - a table, “a professor teaches at most two sections per year”
  - or database, “a student cannot take a course more than once”

Specifying Tables in SQL

```
create table STUDENT (
      SId int not null,
      SName varchar(10) not null,
      MajorId int,
      GradYear int,

      primary key (SId),
      foreign key (MajorId) references DEPT
            on update cascade
            on delete set null,
      check (SId > 0),
      check (GradYear >= 1863)
)
```

Figure 2-4
The SQL specification of the STUDENT table

References

- Edward Sciore, Database Design and Implementation, Wiley, 2009.
  - Chapter 1 & 2